EduPace - Product Requirements Document

Version 1.0 - 17/11/2025

1. Overview

EduPace is a training tool for external pacemaker configuration, combining a physical simulator and a computer-based interface. It allows nurses to safely practice configuring the Medtronic 53401 Single-Chamber Temporary External Pacemaker, replicating the tactile and visual feedback of the real device.

This new development phase builds on the 2024 prototype by Group 8 at TU Delft, addressing its limitations in realism, usability, and reliability. The updated EduPace will integrate new hardware (Arduino GIGA R1 + GIGA Display Shield) and a fully redesigned software architecture, making it a realistic and robust trainer for actual CCU use.

2. Problem Statement

At Renier de Graaf Hospital in Delft, the Medtronic 53401 is used only a few times a year, which causes nurses to not have sufficient routine practice with the device. Current educational materials lack a tactile interface or realistic feedback. This results in low confidence and higher error risk in acute pacing situations. EduPace addresses this by providing a physical practice device coupled to realistic software simulation that reproduces intrinsic rhythms and pacing thresholds.

The first-generation EduPace (2024) successfully demonstrated concept feasibility but lacked clinical realism:

- ECG rhythms were static and artificial, without PVCs, PACs, or realistic variations.
- The interface lacked feedback, sound cues, and scenario dynamics.
- Hardware (Arduino Mega + separate display modules) suffered from limited speed, low resolution, and fragile wiring.
- No persistence or progress tracking for training.

Reinier de Graaf Hospital nurses confirmed that while the idea is promising, it is not yet usable for real training.

3. Objectives

The goal of this new iteration is to transform EduPace into a clinically relevant, modular, and intuitive simulator by improving realism, usability, and maintainability.

Educational and Clinical Objectives:

- Recreate the essential ventricular pacing functions of the Medtronic 53401.
- Provide a safe and realistic environment to practice setting Rate, Output, and Sensitivity parameters.
- Simulate realistic ECG signals that respond dynamically to pacing, including arrhythmias (PVCs, PACs) and incorrect settings.
- Adding auditory and visual feedback for realistic pacing errors.
- Offer feedback and evaluation to reinforce correct pacing technique.

Technical and Development Objectives:

- Replace outdated hardware with a modern, integrated microcontroller (Arduino GIGA R1 WiFi) and touchscreen display (GIGA Display Shield).
- Redesign all software and user interface components for intuitive and interactive operation, reducing setup complexity.
- Implement a dynamic ECG generation engine with adjustable parameters and realistic physiological responses.
- Integrate auditory and visual feedback systems for more immersive training and error awareness.
- Enable training session management and performance tracking, allowing users and instructors to review progress and outcomes.

4. User Needs and Use Cases

4.1 User Needs

- Hands-on training: Nurses must be able to physically adjust the pacing parameters and observe the effect on an ECG
- Realistic feedback: The tool must visually and audibly simulate pacing outcomes and alarms.
- Users must experience and correct incorrect settings safely
- Performance insight
- Accessibility: Simple setup process, intuitive design

4.2 Primary Use Cases

We choose 5 most important, clinically relevant skills that CCU nurses must practice when using a single-chamber ventricular temporary pacemaker like the Medtronic 53401:

UC-01: Initial Setup for Symptomatic Bradycardia (VVI Mode)

The user is presented with a patient with symptomatic ventricular bradycardia. They configure the pacemaker in VVI mode by selecting an appropriate pacing rate above the intrinsic rhythm, choosing a safe initial output (e.g. 5–10 mA), selecting a reasonable sensitivity value, and confirming stable ventricular capture and appropriate sensing on the ECG.

UC-02: Ventricular Capture Threshold Determination

The user increases the pacing rate above the intrinsic rhythm so that all beats are paced, then gradually reduces the output until loss of capture appears. They determine the minimal output level that produces consistent capture and then apply an appropriate safety margin (e.g. 2–3× the threshold) for the final output setting.

UC-03: Ventricular Sensing Threshold Determination & Undersensing

The user sets the pacing rate below the intrinsic rhythm and lowers the output to avoid competitive pacing. They then adjust the Sensitivity setting to detect R-waves reliably, identify undersensing when pacing occurs despite intrinsic QRS complexes, and correct it by increasing sensitivity (lower mV) while maintaining an adequate safety margin.

UC-04: Oversensing & Inappropriate Inhibition of Pacing

The simulator presents a situation where the pacemaker oversenses (e.g. noise, T-waves, or far-field signals), leading to inappropriate inhibition of pacing and bradycardic pauses. The user must recognize the oversensing pattern on the ECG and SENSE indicator and correct it by decreasing sensitivity (higher mV) until pacing resumes appropriately without reintroducing undersensing.

UC-05: Loss of Capture due to Threshold Drift or Lead Issues

During ongoing ventricular pacing, the simulator gradually increases the capture threshold to mimic threshold drift or lead-related problems, resulting in intermittent loss of capture. The

user must recognize the evolving loss of capture, reassess the capture threshold, and increase the output to restore stable capture, understanding that thresholds can change over time and require reassessment.

UC-06: PVCs and PACs Recognition During Pacing

Requested by nurses

UC-07: Wide QRS vs Narrow QRS Interpretation

Requested by nurses

5. System Architecture

5.1 Hardware Architecture

- Microcontroller: Arduino GIGA R1 WiFi
- Display: 3.97” Arduino GIGA Display Shield (800×480 IPS touchscreen)
- Inputs: 3 rotary encoder knobs (Rate, Output, Sensitivity) + Lock button + Optional power button
- Power: USB-C
- Enclosure: 3D-printed case, handheld, ergonomic design, lightweight, durable

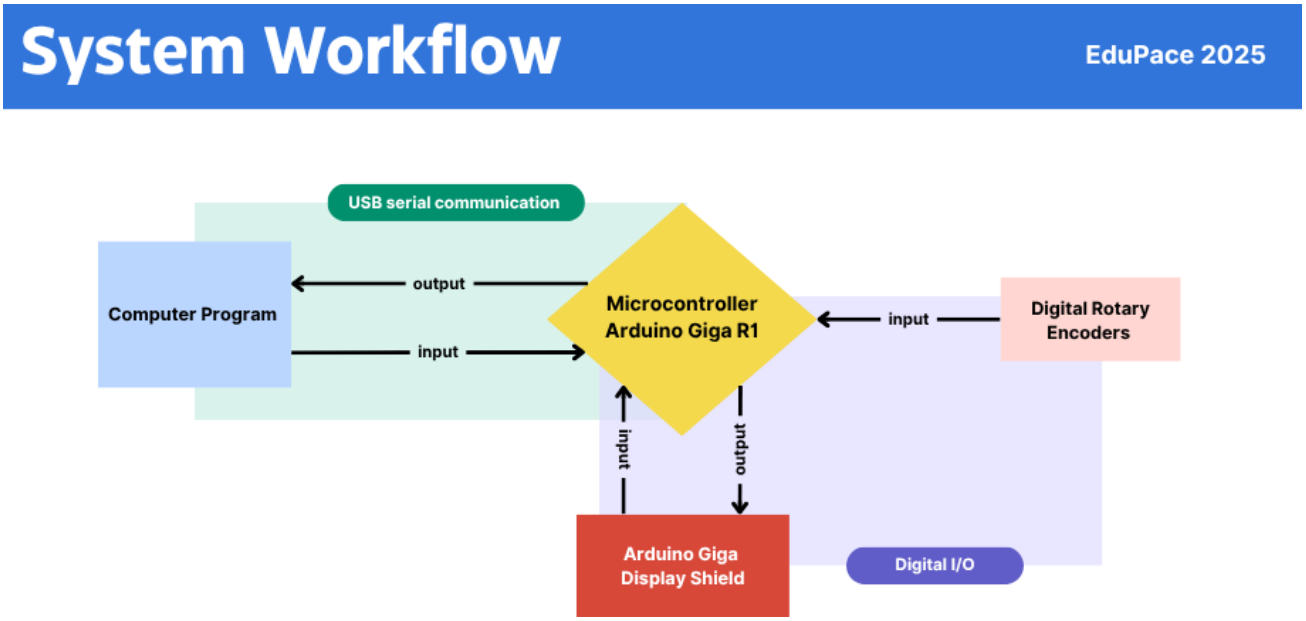
5.2 Software Architecture

- ECG Engine:
  - Mathematical Model producing P-QRS-T complexes
  - Variable morphology and adjustable QRS width
  - Real-time pacing response
  - Noise, PVCs, PACs, loss of capture
- Pacing Model:
  - Synchronous sensing
  - Asynchronous sensing
  - Dynamic capture logic using threshold simulation
- UI Layer:
  - Touchscreen menus on device
  - Real-time ECG display
  - Parameter indicators
  - LED-like pacing / sensing indicators
  - Alarms and warnings
- Communication Layer (Web Serial API)

- Web Serial API for communication between simulator engine and handheld Arduino unit
- Provides a plug-and-play USB interface across Windows, macOS, Linux, and ChromeOS
- No drivers required
- No installation required
- No user configuration required by user

- Training Module
  - Scenario selection
  - Error detection such as undersensing, oversensing, loss of capture, unsafe rate settings.
  - Performance scoring and session summary
  - Export performance log for analysis

5.3 Data Flow and Communication



EduPace uses a bidirectional USB-Serial communication protocol between the handheld hardware unit and the computer-based simulation engine. The hardware will provide the physical controls and on-device indicators, while the computer performs all pacing logic, ECG generation, scenario control.

A. Upstream Data Flow (Hardware → Computer)

The Arduino sends continuously user input and device-state information to the computer. These are:

- Rotary encoder values (Rate knob position, output knob position, sensitivity knob position)
- Lock button state
- Optional touchscreen interactions
- Timestamp

These inputs will be transmitted at a fixed rate and received by the simulation engine wihch uses them to update pacing behaviour and ECG generation in real time.

B. Simulation Logic (Computer side)

The computer application is the central processing unit for the simulator. It will perform:

- Pacing logic (VVI timing, inhibition, sensing, capture, oversensing, undersensing)
- ECG generation
- Scenario engine
- Error detection
- Rendering of ECG waveform, indicators, alarms

The simulation runs independently of the hardware's computing resources, ensuring consistent behaviour across devices.

C. Downstream Data Flow (Computer → Hardware)

The computer sends real-time commands to the Arduino so that the physical device reflects simulator state. These commands include:

- PACE event flag (triggers blink animation on hardware)
- SENSE event flag

This ensures that the handheld hardware mirrors the pacing activity of the simulated pacemaker. This maintains the realism of a physical device but offloads all complex computation to the computer.

D. Communication Protocol

Communication is performed over USB-serial using a lightweight packet protocol

- Upstream packets: structured encoder values and button states
- Downstream packets: pacing / sensing flags
- Heartbeat packet every 100ms – 200ms for connection health
- Automatic reconnection if device is unplugged and reconnected

E. Offline and plug and play operation

All communications are local over USB-C and does not require network connection.

6. Physical Design Specifications

Dimensions	Must be handheld
Weight	Target < 400g
Casing Material	PLA?
Front Panel	3 rotary encoders with tactile detents, touchscreen
Access Ports	USB-C

7. Key Features

Hardware Features

- Rotary encoders with tactile detents for realistic knob control
- Built-in touchscreen
- Compact, durable casing
- USB-C connectivity for single-cable setup (power and data)

Software Features

- Dynamic ECG generation with real-time response
- Scenario selection
- Visual and auditory alarms
- End-of-session performance summary
- Debug mode for demonstrations without hardware connection

8. Technical Requirements

Functional requirements

- The system must simulate accurate pacing logic (VVI) that responds to user input
- The ECG engine must render a physiologically realistic morphology
- The handheld UI must display Rate, Output, Sensitivity, SENSE and PACE indicators
- The system shall simulate oversensing, undersensing, PVCs, PACs, and threshold drift.
- The training module shall record and evaluate user performance.

Non-functional requirements

- Usability: Setup and launch within 45 seconds.
- Maintainability: Modular codebase for future modes and scenarios
- Reliability: <1 crash every 5 continuous hours

Ethical and Safety Requirements

- No patient data is processed or stored
- Requires clear labelling as a training device and not a medical device
- Electrical safety
- Instruction manual included with usage guidelines

9. Deployment and User Operation

Edupace is designed to functioned as an intuitive training tool that clinical staff can launch without technical computer knowledge. The system requires only the EduPace hardware unit and a computer with a modern web browser (Chrome, Edge, Safari). The underlying technical components such as serial communication, Python bridge, and static web hosting, should be abstracted. EduPace is deployed on a lab PC or laptop through a lightweight software bundle. Once installed by technical staff, no further configuration is required for end users.

10. Success Metrics

Usability

- Users can successfully complete all five training scenarios without guidance
- Setup time should be less than 45 seconds
- Device should be plug and play and not require any technical skill.

Technical

- < 1 crash per 5 hours

11. Validation and Testing Plan

Hardware Testing

- Durability testing (3D printed case,

Software Testing

- ECG model validation

User Evaluation

- Sessions with CCU nurses (Rob)
- Feedback sessions with clinical coordinator (Paul)
- Feedback sessions with project coordinator (John)

12. Future Development

- Dual-chamber pacing
- Additional arrhythmias
- Instructor dashboard
- Wireless communication

13. Appendices